CYBERSECURITY AVANZATA · LEZIONE 3 DI 8

Password, hash e salt

Come deve conservare le password un server fatto bene, perché le fughe di dati diventano liste di password in chiaro, e come si controlla una password senza rivelarla.

5G TPSI

SHA-256, salt, PBKDF2, bcrypt, Argon2, k-anonimato, credential stuffing

PER COMINCIARE

Se un sito subisce una fuga di dati, che cosa pensi che trovino gli attaccanti al posto della tua password?

SONDAGGIO

- **A** La password in chiaro
- **B** La password cifrata con una chiave
- **C** Un'impronta (hash) da cui non si torna indietro
- **D** Dipende dal sito

RIPASSO VELOCE

Che cosa fa una funzione di hash crittografica

- **Deterministica**: stesso input, stessa impronta
- **Lunghezza fissa**: SHA-256 dà sempre 256 bit, 64 cifre esadecimali
- **A senso unico**: dall'impronta non si ricava l'input
- **Effetto valanga**: cambi un carattere, cambia tutto
- **Resistente alle collisioni**: non si trovano due input con la stessa impronta

`SHA-256("ciao")`

b133a0c0e9bee3be20163d2ad31d6248db292aa6dcb1ee087a2aa50e0fc75ae2

`SHA-256("Ciao")`

25c73520e69f4bf229811e8e46ffe7d80471544b9bee15ed25044b86be4115ad

AL COMPUTER · 15 minuti

SHA-256 in Java

```
MessageDigest md = MessageDigest.getInstance("SHA-256");
byte[] h = md.digest(testo.getBytes(StandardCharsets.UTF_8));
String hex = HexFormat.of().formatHex(h);
```

1. Scrivi **Impronta.java** che legge una parola dalla tastiera e stampa il suo SHA-256.
2. Prova `ciao` e `Ciao` e confronta con la slide precedente.
3. Misura con `System.nanoTime()` quanto tempo serve per calcolare **un milione** di hash.

Controllo: `ciao` inizia con **b133a0c0**. Un milione di SHA-256 richiede di solito meno di un secondo su un PC del laboratorio.

QUIZ A TRABOCCHETTO

Per conservare le password, qual è la funzione migliore tra queste?

QUIZ

A SHA-512, perché è la più lunga

B SHA-256, perché è lo standard di oggi

C AES-256, perché è cifratura militare

D PBKDF2 con 600.000 iterazioni e un salt

IL CONTO DELL'ATTACCANTE

Quanto ci mette a provarle tutte?

Password	Combinazioni	Hash veloce, 10 miliardi al secondo	Hash lento, 10.000 al secondo
8 minuscole	$26^8 \approx 2{,}1 \cdot 10^{11}$	**21 secondi**	circa 8 mesi
8 tra maiuscole, minuscole, cifre	$62^8 \approx 2{,}2 \cdot 10^{14}$	**6 ore**	circa 700 anni
12 minuscole	$26^{12} \approx 9{,}5 \cdot 10^{16}$	**110 giorni**	circa 300.000 anni

> Ipotesi di esempio, caso peggiore (tutte le combinazioni). E nessuno parte dalla forza bruta: si parte dai **dizionari** di password già rubate.

IL SALE

Salt: stessa password, impronte diverse

Utente	Salt (casuale)	hash(salt + password)
anna	`9f2c…`	`41d7…`
marco	`03ab…`	`e85c…`

Anna e Marco hanno la stessa password, ma nel database non si vede.

- Il salt è casuale, diverso per ogni utente, e si salva **in chiaro** accanto all'hash
- Rende inutili le **tabelle precalcolate** (rainbow table)
- Obbliga l'attaccante a ricominciare da capo per ogni utente
- Non serve tenerlo segreto: serve che sia unico

PUZZLE

Metti in ordine: il server registra una nuova password

1. Cancella la password dalla memoria e non la scrive nei log
2. Salva algoritmo, parametri, salt e hash nel database
3. Riceve la password via HTTPS
4. Calcola l'hash lento di salt e password
5. Genera un salt casuale con SecureRandom
6. Controlla lunghezza e che non sia in una lista di password rubate

AL COMPUTER · 25 minuti

PBKDF2 nel JDK: senti la lentezza

```
byte[] salt = new byte[16];
new SecureRandom().nextBytes(salt);
PBEKeySpec spec = new PBEKeySpec(pw, salt, 600_000, 256);
byte[] h = SecretKeyFactory
  .getInstance("PBKDF2WithHmacSHA256")
  .generateSecret(spec).getEncoded();
```

1. In **Cassaforte.java** scrivi `registra(char[] pw)` e `verifica(char[] pw, salt, hash)`.
2. Misura il tempo di una chiamata. Poi stampa due volte l'hash della stessa password.

Controllo: una chiamata dura da qualche decina a qualche centinaio di millisecondi. Le due impronte della stessa password sono **diverse**, ma `verifica` risponde **true** per entrambe.

TRE FAMIGLIE LENTE

bcrypt, scrypt, Argon2

bcrypt

1999. Un parametro di costo: ogni +1 raddoppia il tempo. Diffusissimo, accetta al massimo 72 byte di password.

scrypt

2009. Lento e anche **affamato di memoria**: le schede grafiche hanno tanti core ma poca memoria per ciascuno.

Argon2id

Vincitore della Password Hashing Competition (2015). Regoli tempo, memoria e parallelismo. La prima scelta oggi.

La regola pratica: **non inventare**. Usa la funzione della libreria del tuo framework, con i parametri raccomandati dell'anno in corso.

IL RIUSO

Credential stuffing: la tua password di un altro sito

Le password rubate a un sito finiscono in liste con miliardi di coppie **e-mail · password**. Gli attaccanti le provano in automatico su tutti gli altri siti.

Non serve rompere niente: basta che tu abbia riusato la stessa password.

Le difese

- Utente: password manager, una password diversa per sito, MFA
- Server: limite ai tentativi, blocco degli IP sospetti, controllo contro le liste di password rubate

AL COMPUTER · 20 minuti

È già stata rubata? Chiedilo senza dirla

1. Calcola lo **SHA-1** della password di prova `password`, in esadecimale maiuscolo (cambia l'algoritmo nel programma di prima).
2. Prendi i primi **5 caratteri** e chiedi:
   `curl -s https://api.pwnedpasswords.com/range/XXXXX`
3. Nella risposta cerca il resto dell'impronta, dopo i primi 5 caratteri.
4. Ripeti con `Novaula-Lab-2026!` e annota in **password-cognome.md** i due risultati.

Controllo: l'impronta è **5BAA61E4C9B93F3F0682250B6CF8331B7EE68FD8**; la riga **1E4C9B93…** è seguita da un numero di diversi milioni. La seconda password, con buona probabilità, non c'è.

UNA PREVISIONE DEL 2004

La fine delle password, rimandata da vent'anni

> **Non c'è dubbio che, col tempo, le persone si affideranno sempre meno alle password.**

Bill Gates, RSA Conference, febbraio 2004, riportato dalla stampa dell'epoca



QUIZ

Un sito ti obbliga a cambiare password ogni 30 giorni, con maiuscola, cifra e simbolo. Che cosa ottiene di solito?

QUIZ

A Password molto più sicure

B Nessun effetto

C Password prevedibili come Estate2026! e Autunno2026!

D Che gli utenti usino il password manager

PUZZLE

Abbina il problema alla difesa

Rainbow table	Password manager e MFA
Miliardi di tentativi al secondo con la GPU	Non registrare mai i campi password
Password riusata su un altro sito	Limite ai tentativi e blocco temporaneo
Tentativi di login a raffica	Funzione lenta come Argon2id
Password in chiaro nei log	Salt casuale per ogni utente

OGGI

L'AI indovina password?

Gli attaccanti non provano combinazioni a caso: usano **dizionari** di password rubate e **regole** (maiuscola all'inizio, anno alla fine, a → @).

Programmi come hashcat applicano migliaia di regole a ogni parola del dizionario.



SFIDA A SQUADRE

Apri il lucchetto



- 1ª e 2ª cifra: quanti byte di salt usa il laboratorio di PBKDF2
- 3ª e 4ª cifra: quante cifre esadecimali ha un'impronta **SHA-1**, come quella del laboratorio Pwned Passwords

codice

BIGLIETTO D'USCITA

Completa prima di uscire

Il valore casuale salvato accanto all'hash si chiama _____.

Per le password servono funzioni di hash volutamente _____.

La funzione vincitrice della Password Hashing Competition è _____.

Provare su un sito le password rubate a un altro si chiama credential _____.

Pwned Passwords riceve solo i primi _____ caratteri dell'hash.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Hash: a senso unico, effetto valanga
- Hash veloci (SHA-256) no per le password
- Salt unico + funzione lenta: PBKDF2, bcrypt, Argon2id
- Confronto a tempo costante, niente password nei log
- Credential stuffing: il riuso è il vero nemico
- k-anonimato: controllare senza rivelare
- Lunghezza e MFA battono le regole rigide
- Prossima lezione: TLS e certificati

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.